

SELECTING WORKER-AGENT TRAINING CONTEXT UNDER CONTINUAL TOOL-USE SHIFT

Anonymous authors

Paper under double-blind review

ABSTRACT

Meta-agents need reliable ways to train and evaluate the worker agents they manage. We study whether a controller can select between two worker-agent training-context policies after each stage of a continual tool-use stream: A strips prior action–observation lines, while B retains them. The controller observes validation metrics only and carries the selected worker adapter forward; final evaluation uses API families and stream orders held out from all controller decisions. The accompanying seed-42 fixed-policy pilot finds higher final exact full-call accuracy for B (56.9% versus 39.2%), but also more malformed/no-call outputs and a 25.1% larger token budget. We therefore frame the fixed-policy result as motivation and the controller protocol as the main meta-agent contribution, with reliability-aware selection and explicit safeguards against test leakage.

1 INTRODUCTION

A meta-agent that trains or improves another agent needs feedback about more than a final scalar score. For tool-use agents, an interaction exposes a structured action–observation history: the request made to an API, the returned observation, and the next action selected by the agent. These traces are externally auditable records of worker behavior, not private chain-of-thought.

We ask: *under a fixed token budget, can a controller select between stripped-context and trajectory-context training for a continually adapting tool-use worker, using only validation signals, and does that selection generalize to unseen API families and stream orders?* This is a component-level instance of meta-agent post-training: the controller makes a policy decision about how another agent should learn.

The worker policies are A, which removes prior API request and response lines from the prompt, and B, which preserves them. Both predict the next API call; this is a context-policy comparison, not a pure final-response-only versus process-supervision comparison. API families are assigned to disjoint fit, controller-validation, and final-test partitions. After each stream stage, the controller trains both candidates from the currently selected adapter, evaluates them on validation families, and carries the higher-scoring adapter to the next stage. The final test families are not used for selection, score design, or stream-order tuning.

We also retain the earlier fixed-policy pilot as diagnostic evidence. It shows why a controller should use decomposed reliability metrics: B greatly reduces wrong-API errors but increases malformed/no-call outputs, and its original run receives 25.1% more training tokens. The new protocol addresses the budget issue for candidate training and makes the meta-level decision auditable.

Our contributions are:

- A controller protocol that selects between two worker-agent training-context policies after each continual-learning stage using validation metrics only.
- A leakage-resistant evaluation design with disjoint API-family fit, controller-validation, and final-test partitions plus held-out stream orders.
- A reliability-aware selection score that separates tool selection, exact arguments, valid-call rate, malformed/no-call behavior, and wrong-API behavior.
- A diagnostic fixed-policy comparison showing why meta-agents optimizing only aggregate exact match can select an unreliable worker policy.

The study is deliberately component-level. We do not claim to introduce a complete self-improving meta-agent, and we do not use private reasoning traces. Controller results are not available until the GPU experiment is run; the fixed-policy pilot is reported separately as motivation and diagnostic evidence.

2 RELATED WORK

Tool-use agents and trajectories. Toolformer showed that language models can learn when and how to call external tools (Schick et al., 2023). API-Bank provides structured dialogues containing API requests, responses, and assistant actions (Li et al., 2023); ToolLLM and ToolBench scale tool-use supervision to substantially larger API collections (Qin et al., 2023). Our setting uses API-Bank because its compact size permits repeated sequential fine-tuning and complete generation evaluation.

Process and outcome supervision. Process supervision can outperform final-answer supervision in mathematical reasoning (Lightman et al., 2023). API trajectories are a narrower signal: they record observable interaction events rather than internal reasoning steps or human labels. We therefore use the term trajectory context and avoid treating API traces as faithful representations of thought.

Continual learning and meta-level optimization. Sequential fine-tuning can degrade earlier capabilities, motivating evaluation protocols that measure adaptation and forgetting (Wang et al., 2023). Our evaluation matrix follows this perspective. Efficient adaptation methods such as LoRA and QLoRA make it practical to train multiple worker-agent variants (Hu et al., 2021; Dettmers et al., 2023). A controller that trains and compares these variants is a concrete instance of the post-training meta-agent studied here.

3 WORKER-AGENT SETUP

3.1 TASK AND STREAM CONSTRUCTION

We use API-Bank, a dataset of tool-augmented dialogues, and construct four sequential blocks $D_1, \dots, D_4$ by grouping examples by API family. The blocks are disjoint in their assigned examples. Within each block, API families are deterministically assigned to fit, controller-validation, or final-test roles. Fit uses only original training examples; validation and test use only original held-out examples, and no API family appears in more than one role. The controller is run under canonical, reverse, and one-block-rotated stream orders. At each stage it trains candidates on fit families and observes validation metrics only. Final test-family evaluation occurs after all selections and is never used for score design or policy selection. The controller’s validation history is stage-indexed, recording stage-wise validation metrics rather than producing a 4×4 matrix. Rows index the training stage and columns index the evaluation block.

The primary generation task is next-API-call prediction. We score examples whose expected output contains a parseable API call. The full held-out evaluation contains 126, 104, 103, and 107 scored examples in D_1 through D_4 , respectively. The training notebook also reports a sampled evaluation of 32 examples per block at each stage; we report it separately because it is useful for continual-learning metrics but has higher sampling noise.

3.2 TRAINING CONDITIONS AND CONTROLLER

The stripped-context condition, A, removes previous lines beginning with `API-Request:` or `API-Response:`, as well as the corresponding trace markers, before training and generation evaluation. It is a stripped-context next-call baseline, not a pure final-response-only condition: both conditions predict the next API call when the target is an API call.

The trajectory condition, B, retains the previous API requests and responses in the prompt. Generation evaluation uses the same condition-specific format as training. This alignment matters: evaluating B with stripped prompts would create a train/test mismatch and confound the comparison.

Table 1: Full held-out generation results after sequential training through D_4 . Values are percentages.

Condition	Metric	D1	D2	D3	D4	Mean
A	Exact full call	35.7	43.3	32.0	45.8	39.2
B	Exact full call	57.9	61.5	44.7	63.6	56.9
A	API name	64.3	62.5	60.2	79.4	66.6
B	API name	73.8	82.7	67.0	73.8	74.3
A	Name + any parameter	51.6	56.7	50.5	65.4	56.1
B	Name + any parameter	67.5	76.9	61.2	72.0	69.4

At stage t , both candidate adapters W_t^A and W_t^B are trained from the previously selected adapter W_{t-1} on the current block’s fit families. The controller evaluates each candidate on validation families from blocks observed through stage t and selects $W_t = \arg \max_{c \in \{A, B\}} S(V_t^c)$. The score is frozen before testing:

$$S(V) = 0.50a_{\text{exact}} + 0.20a_{\text{name}} + 0.20r_{\text{valid}} - 0.05r_{\text{malformed}} - 0.05r_{\text{wrong-API}}.$$

Candidate training is token-matched by capping B to the number of truncated training tokens used by A for the same fit families. The controller’s decision history, candidate scores, and selected adapter are saved for audit. All runs use `meta-llama/Llama-3.1-8B-Instruct`. We use QLoRA with 4-bit NF4 quantization, LoRA rank 32, alpha 64, bfloat16 computation, maximum sequence length 1024, effective batch size 16, learning rate 2×10^{-4} , and three epochs per block. The recorded run uses seed 42. The A run used an NVIDIA RTX PRO 6000 Blackwell Server Edition and the B run used an NVIDIA H100 80GB; both have sufficient memory for the configuration.

The earlier fixed-policy pilot was not token-budget matched: A consumed 1,857,169 training tokens and B consumed 2,324,314, so B received 25.1% more tokens. The controller experiment uses token-matched candidate training; we report the pilot separately from controller results.

3.3 METRICS

For each generated call, we parse the API name and single-quoted parameter assignments, then normalize names and parameter strings for comparison. We report:

- **API-name accuracy:** the generated API name matches the expected name.
- **Exact full-call accuracy:** the API name and normalized parameter dictionary match exactly.
- **Name plus any parameter:** the API name and at least one expected parameter-value pair match.
- **Malformed/no-call rate:** no parseable API call is generated.

For the sampled continual-learning matrix, we also calculate final average accuracy (AA), backward transfer (BWT), forward transfer (FWT), average forgetting, and area under the learning curve (AULC). These metrics are descriptive for this one seed; they are not accompanied by uncertainty estimates.

4 RESULTS

4.1 FULL HELD-OUT GENERATION

Table 1 reports the final-stage results after training through D_4 . B improves exact full-call accuracy on every held-out block, with a mean increase of 17.7 percentage points. The gain in API-name accuracy is smaller but consistent in the mean, at 7.7 points. Name-plus-any-parameter accuracy also improves on every block.

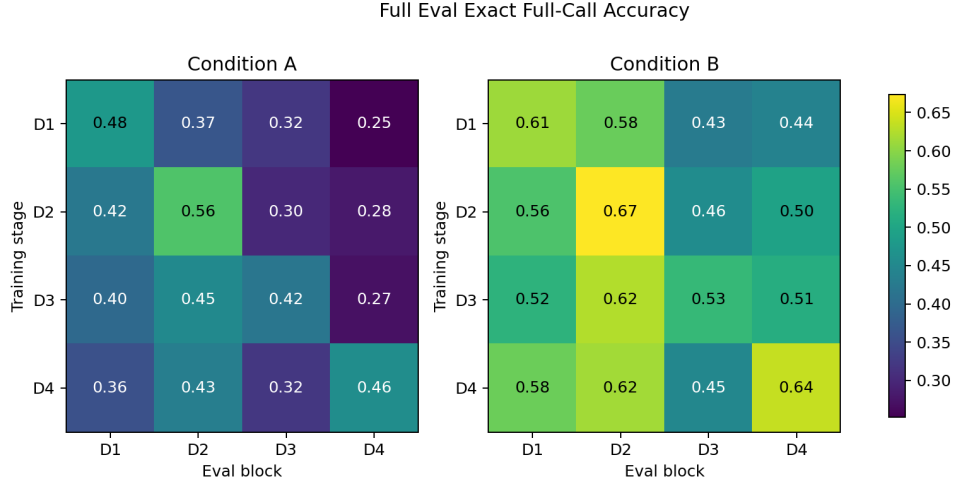


Figure 1: Full held-out exact full-call accuracy matrices. Rows are training stages and columns are evaluation blocks. The trajectory condition is higher on every final-stage block, while earlier-block performance still changes over the stream.

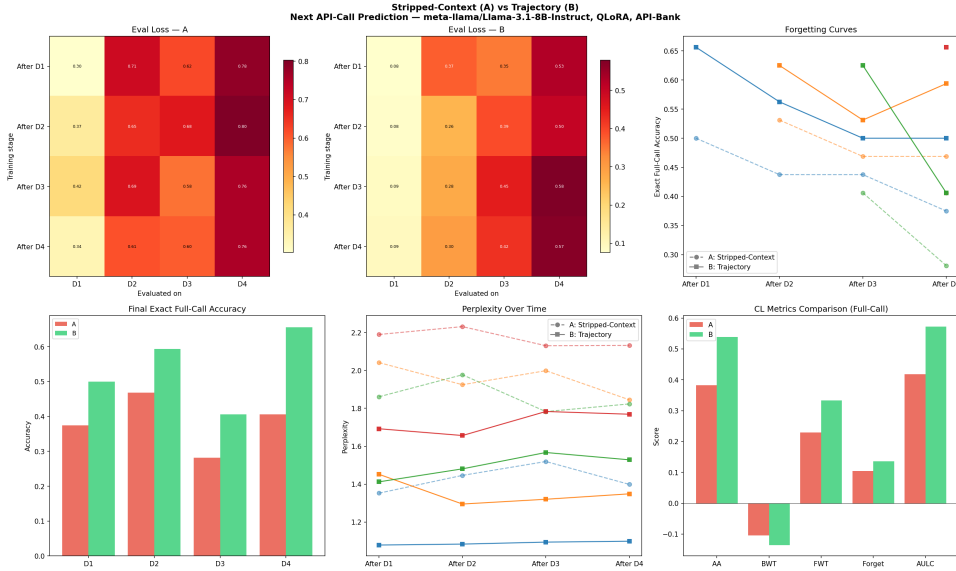


Figure 2: Sampled continual-learning evaluation. B has higher final average accuracy and forward transfer, but both conditions show forgetting and B’s sampled retention metrics are more negative.

4.2 CONTINUAL-LEARNING BEHAVIOR

The sampled evaluation gives final AA of 38.3% for A and 53.9% for B. B also has higher FWT (33.3% versus 22.9%) and higher AULC (57.2% versus 41.8%). These results are consistent with faster or stronger adaptation to the stream, although they are measured on a small fixed sample and B receives more tokens.

The forgetting metrics point in the opposite direction. BWT is -13.5% for B and -10.4% for A, while average forgetting is 13.5% for B and 10.4% for A. Thus, the trajectory condition should not be described as reducing forgetting in this experiment. The more accurate conclusion is that it improves final accuracy and forward transfer while not improving, and possibly worsening, the measured retention tradeoff.

Table 2: Final-stage error categories across all scored blocks. Counts are mutually exclusive under the evaluation parser.

Category	A	B
Exact full call	172	251
Correct API, some parameters	74	54
Correct API, wrong parameters	47	22
Wrong API	102	12
Malformed or no call	45	101

4.3 FAILURE MODES

The aggregate gain is driven partly by tool selection. Across all final-stage scored examples, A produces 102 wrong-API errors and B produces 12. B also reduces correct-API/wrong-parameter errors from 47 to 22. However, B produces 101 malformed or no-call outputs versus 45 for A. This means a supervisor using only exact-call accuracy could miss a reliability regression in output formatting or call emission.

For a meta-agent, this decomposition is operationally important. A controller that optimizes only exact-match reward could prefer a policy that selects the right tool more often but emits more invalid actions. Any automatic optimizer should therefore use multiple constraints or reward terms, including parse validity, semantic argument correctness, and task completion.

5 IMPLICATIONS FOR META-AGENT SYSTEMS

The implemented experiment instantiates a narrow meta-agent: a controller that trains, evaluates, and selects a worker adapter. At each stage it compares A and B from the same parent adapter, observes validation metrics, and selects the candidate carried to the next stage. Its final evaluation is restricted to API families held out from both fitting and controller validation.

The controller tests whether validation-based selection generalizes across stream orders. It uses a frozen score containing exact-call accuracy, API-name accuracy, valid-call rate, malformed/no-call rate, and wrong-API rate. The saved history exposes each candidate score and selected policy, making the decision auditable without exposing final-test metrics.

The fixed-policy pilot suggests that a useful controller objective should not be a single exact-call metric. The trajectory condition’s reduction in wrong-tool errors is valuable, but its malformed/no-call increase is a warning that optimizing one observable metric can move failures elsewhere. The controller therefore uses a frozen multi-metric score.

The implemented controller experiment adds this selection loop under equal candidate token budgets. It evaluates the selected worker only on held-out API families under canonical, reverse, and rotated stream orders; controller decisions use validation families only. The fixed-policy pilot is reported as observed evidence; controller outcomes are a separate experiment and must be added only after the GPU run, together with its split manifest and decision history.

6 LIMITATIONS

Token budget and seed. B receives 25.1% more training tokens, and the reported run uses only seed 42. The observed advantage may therefore reflect token quantity, trajectory content, or their interaction. No confidence intervals or significance tests are reported.

Task and data scope. API-Bank is compact and structured, which supports controlled iteration but limits external validity. Four blocks are a short continual-learning stream. The task is next-call prediction rather than complete end-to-end task success.

Metric brittleness. Exact parameter matching and regular-expression parsing can classify semantically equivalent calls as failures. Conversely, API-name correctness does not guarantee a useful or

safe action. Future evaluations should include semantic argument scoring and executable task-level outcomes.

Scope of meta-agent. The controller is deliberately narrow: it selects between two human-designed context policies and does not perform open-ended architecture search, self-rewriting, multi-agent delegation, or human management. Its contribution is an auditable post-training and evaluation component, not unrestricted self-improvement.

7 RESPONSIBLE-USE STATEMENT

Tool-use post-training can increase an agent’s ability to select and invoke external APIs, so improvements should not be treated as permission to grant unrestricted access. The trajectories studied here are observable interaction records, not private reasoning, but they may still contain sensitive user or tool-output data; deployments should minimize collection, remove unnecessary identifiers, and apply access controls. Any meta-agent that trains or selects worker policies should operate in a sandbox with least-privilege tools, explicit human approval for consequential actions, complete audit logs, rate limits, and a rollback or halt mechanism. Evaluation should separately monitor invalid calls, unsafe tool selection, semantic argument errors, and metric gaming rather than optimizing exact-match accuracy alone. We recommend that future trajectory-optimization studies release only permitted, de-identified data and document dataset licenses and model-use restrictions.

8 CONCLUSION

We introduced a narrow meta-controller that selects between stripped-context and trajectory-context training for a continually adapting tool-use worker. It trains both candidates at each stage, selects using a frozen reliability-aware score on controller-validation API families, and evaluates only after training on final-test families and held-out stream orders. The fixed-policy seed-42 pilot motivates the design: trajectory context reduced wrong-API errors but increased malformed/no-call outputs and used more tokens.

The claim remains component-level. The controller does not establish open-ended self-improvement, and controller results should be reported only with the split manifest, token accounting, and all stream-order outcomes. Multiple seeds, larger streams, semantic task scoring, and comparisons against learned or human-designed controller baselines remain necessary to establish robust selection generalization.

REFERENCES

- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *NeurIPS*, 2023.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. *arXiv:2304.08244*, 2023.
- Hunter Lightman et al. Let’s verify step by step. *arXiv:2305.20050*, 2023.
- Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv:2307.16789*, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.
- Xiao Wang et al. TRACE: A comprehensive benchmark for continual learning in large language models. *arXiv:2310.06762*, 2023.

A REPRODUCIBILITY DETAILS

The repository contains the numbered notebooks used for data preparation, training/evaluation, analysis, full generation evaluation, and the meta-controller experiment. The canonical pilot uses seed 42 and saves result JSON files for both fixed conditions. `final/05_meta_controller.ipynb` creates disjoint fit, controller-validation, and final-test API-family partitions, trains both candidates after each stage, selects from validation metrics only, and evaluates the selected worker on canonical, reverse, and rotated stream orders. The split manifest and controller history are saved under `meta_controller_seed42/`. A smoke test checks that condition A strips trajectory lines and condition B preserves them in both training and evaluation formats. Large adapter bundles and full-evaluation outputs are stored outside git; the repository includes manifests and partial metadata for verification. The reported hardware and run records are documented in the accompanying run ledger.